Active Record Associations

This guide covers the association features of Active Record.

1 Why Associations?

Why do we need associations between models? Because they make common operations simpler and easier in your code. For example, consider a simple Rails application that includes a model for customers and a model for orders. Each customer can have many orders. Without associations, the model declarations would look like this:

```
class Customer < ActiveRecord::Base
end

class Order < ActiveRecord::Base
end
```

Now, suppose we wanted to add a new order for an existing customer. We'd need to do something like this:

```
@order = Order.create(order_date: Time.now, customer_id: @customer.id)
```

Or consider deleting a customer, and ensuring that all of its orders get deleted as well:

```
@orders = Order.where(customer_id: @customer.id)
@orders.each do |order|
  order.destroy
end
@customer.destroy
```

With Active Record associations, we can streamline these - and other - operations by declaratively telling Rails that there is a connection between the two models. Here's the revised code for setting up customers and orders:

```
class Customer < ActiveRecord::Base
  has_many :orders, dependent: :destroy
end

class Order < ActiveRecord::Base
  belongs_to :customer
end
```

With this change, creating a new order for a particular customer is easier:

```
@order = @customer.orders.create(order_date: Time.now)
```

Deleting a customer and all of its orders is *much* easier:

```
@customer.destroy
```

To learn more about the different types of associations, read the next section of this guide. That's followed by some tips and tricks for working with associations, and then by a complete reference to the methods and options for associations in Rails.

2 The Types of Associations

In Rails, an *association* is a connection between two Active Record models. Associations are implemented using macro-style calls, so that you can declaratively add features to your models. For example, by